Gestock

Planejamento do Projeto

Plataforma web de gestão de estoque e previsão de demanda para pequenos estabelecimentos. Visão, stack, arquitetura, segurança, modelo de dados, previsão e roadmap do MVP.

Projeto de Extensão · Versão inicial · novembro de 2026

> Plataforma web de gestão de estoque e previsão de demanda para pequenos estabelecimentos. Primeiro cliente de validação: uma açaiteria parceira (sem uso da marca dela no produto).

Este documento é vivo: toda decisão importante entra aqui (ou em `docs/decisoes/`) antes de virar código.

1. Visão em uma frase

Transformar vendas e movimentações de estoque em **indicadores, previsões e recomendações de compra**, deixando sempre claro o que é **dado medido** e o que é **estimativa**.

Princípios do produto

1. **Genérico desde o início** — nada de regra "de açaiteria" chumbada no código. Particularidades viram configuração (ex.: "produto vendido por peso").
2. **Multi-tenant** — cada estabelecimento só enxerga os próprios dados, garantido no banco (RLS), não só no código.
3. **Honestidade dos dados** — o sistema nunca inventa ingrediente. Consumo estimado é marcado como estimado; previsão sem histórico suficiente diz isso.
4. **Simples de usar no balcão** — registrar uma venda presencial tem que levar poucos segundos, no celular.

2. Stack escolhida

Camada	Tecnologia	Por quê
Linguagem	**TypeScript** (front e back)	Uma linguagem só, tipos compartilhados, menos bugs.
Monorepo	**pnpm workspaces**	Front, back e pacote compartilhado no mesmo repositório, um único `install`.
Front-end	**React + Vite**	Rápido de desenvolver e de buildar; ecossistema enorme.
UI	**Tailwind CSS + shadcn/ui**	Componentes prontos e acessíveis, fácil aplicar o tema azul-marinho/branco.
Estado de servidor	**TanStack Query**	Cache, loading e refetch sem boilerplate.
Formulários	**React Hook Form + Zod**	Validação com o mesmo schema usado no back.
Gráficos	**Recharts**	Suficiente para o dashboard.
Back-end	**NestJS** (Node 20)	Estrutura modular padronizada (módulos, controllers, services, guards) — ajuda a manter organização com pouco tempo.
ORM / migrations	**Prisma**	Schema declarativo, migrations versionadas, tipagem automática.
Banco	**PostgreSQL 16**	Relacional (estoque e vendas são dados fortemente relacionais), transações, `numeric` exato, **Row Level Security** nativo, funções de data/agrupamento boas para previsão.
Hospedagem do banco	**Supabase (só o Postgres)** ou **Neon** — plano gratuito	Postgres gerenciado com backup, sem precisar de Docker local. Usamos apenas o banco; auth e API são nossas.
Autenticação	Própria: **Argon2** + **JWT** (access curto + refresh em cookie httpOnly)	Controle total, sem dependência de terceiros.
Testes	**Vitest** (front e unidades) + **Jest/Supertest** (API, padrão Nest)	Testar principalmente regras de estoque, isolamento de tenant e previsão.
Qualidade	**ESLint + Prettier + Husky + lint-staged + commitlint**	Padrão de código e de commit automático.
CI	**GitHub Actions**	Lint, typecheck, testes e build em todo PR.
Deploy (sugestão)	Front: **Vercel** · API: **Render** ou **Railway**	Planos gratuitos/baratos, deploy a partir do GitHub.

> **Por que não Supabase completo (auth + API automática)?** A decisão é desenvolver front e back nós mesmos. Mantemos Postgres + RLS (o que o Supabase tem de melhor para nós) e escrevemos a API em NestJS, o que dá mais aprendizado e controle sobre regras de negócio (baixa de estoque, previsão).

> **Ambiente local atual:** Node 20.18 e npm 10 instalados; sem Docker e sem GitHub CLI. Por isso a sugestão de banco gerenciado. Se quiserem, dá para instalar Postgres local ou Docker depois.

3. Arquitetura

3.1 Estrutura do repositório

```
gestock/
├── apps/
│   ├── web/                # React + Vite (front-end)
│   │   └── src/
│   │       ├── app/        # rotas, layout, providers
│   │       ├── features/   # uma pasta por domínio (auth, produtos, estoque, vendas, dashboard, previsao)
│   │       ├── components/ # componentes genéricos (ui/ = shadcn)
│   │       └── lib/        # cliente HTTP, utils
│   └── api/                # NestJS (back-end)
│       ├── prisma/         # schema.prisma, migrations, seed
│       └── src/
│           ├── common/     # guards, interceptors, filtros de erro, decorators
│           ├── infra/      # prisma service, config, logger
│           └── modules/
│               ├── auth/
│               ├── tenants/        # estabelecimentos e membros
│               ├── catalog/        # produtos, ingredientes, categorias, unidades, receitas
│               ├── inventory/      # movimentações e saldo
│               ├── sales/          # vendas presenciais e online
│               ├── dashboard/
│               └── forecasting/    # previsão e recomendação de reposição
├── packages/
│   └── shared/             # schemas Zod, enums e tipos usados por web e api
├── docs/                   # este arquivo, decisões (ADRs), diagramas
└── .github/workflows/      # CI
```

3.2 Camadas do back-end (dentro de cada módulo)

```
Controller  →  recebe HTTP, valida DTO (Zod), checa permissão
Service     →  regra de negócio (ex.: venda gera baixa de estoque)
Repository  →  acesso ao banco via Prisma (sempre dentro do contexto do tenant)
```

Regra: controller não fala com Prisma; service não conhece HTTP.

3.3 Fluxo de uma requisição

```
Front (JWT no header)
  → API: AuthGuard valida token → extrai user_id e tenant_id ativo
  → TenantGuard confere que o usuário é membro do tenant e tem a permissão
  → Service abre transação e executa  SET LOCAL app.tenant_id = '<uuid>'
  → Postgres aplica RLS: qualquer SELECT/UPDATE só vê linhas desse tenant
```

4. Multi-tenant e segurança

4.1 Modelo de isolamento

- **Banco único, schema único, coluna `tenant_id` em toda tabela operacional.** Mais simples de manter e escalar para muitos clientes pequenos.
- **Defesa em profundidade (duas camadas):**
    1. **Aplicação:** todo repositório filtra por `tenant_id` vindo do token, nunca do body.

2. **Banco (RLS):** políticas `USING (tenant_id = current_setting('app.tenant_id')::uuid)` com `FORCE ROW LEVEL SECURITY`. Mesmo que um bug esqueça o filtro, o banco bloqueia.

- **Dois usuários de banco:**
  - `gestock_owner` — dono das tabelas, usado **só** para rodar migrations.
  - `gestock_app` — usado pela API, **sem** `BYPASSRLS`, sem permissão de DDL.
- Teste automatizado obrigatório: "usuário do tenant A não consegue ler/alterar nada do tenant B".

4.2 Autenticação

- Senha com **Argon2id**.
- **Access token** JWT de ~15 min (no header `Authorization`).
- **Refresh token** de ~7 dias em cookie `httpOnly`, `Secure`, `SameSite=Strict`, com rotação e armazenamento do hash no banco (permite logout/revogação).
- Rate limiting no login e cadastro (`@nestjs/throttler`).
- Bloqueio temporário após várias tentativas erradas.

4.3 Autorização (papéis por estabelecimento)

Papel	Pode
`OWNER`	Tudo, inclusive convidar/remover membros e editar dados do estabelecimento.
`MANAGER`	Catálogo, receitas, estoque, vendas, relatórios, previsão.
`STAFF`	Registrar vendas e entradas de estoque; consultar estoque.

Um usuário pode pertencer a mais de um estabelecimento (tabela `memberships`).

4.4 Outras medidas

- Validação de toda entrada com **Zod** (whitelist; campos extras rejeitados).
- **Helmet**, **CORS** restrito ao domínio do front.
- Segredos só em variáveis de ambiente (`.env` no `.gitignore`; `.env.example` versionado).
- Prisma usa queries parametrizadas (proteção contra SQL injection).
- Logs sem dados sensíveis (sem senha, sem token).
- `audit_log` para ações sensíveis (ajuste de estoque, exclusões, mudança de papel).
- Dependabot ativado no GitHub.
- LGPD: coletar o mínimo de dados pessoais (nome e e-mail dos usuários; nenhum dado de cliente final no MVP).

5. Modelo de dados (MVP)

Convenções: `id uuid`, `created_at`, `updated_at`; quantidades em `numeric(14,3)` na **unidade base** do item (g, ml ou un); dinheiro em `numeric(12,2)`.

5.1 Contas

- **users** — id, name, email (único), password_hash, created_at
- **tenants** — id, name, slug, segment (açaiteria, sorveteria, mercado...), timezone, created_at
- **memberships** — user_id, tenant_id, role (`OWNER|MANAGER|STAFF`), status
- **refresh_tokens** — user_id, token_hash, expires_at, revoked_at

5.2 Catálogo

- **categories** — tenant_id, name, kind (`PRODUCT|INGREDIENT`)
- **units** — código e unidade base (`g`, `kg` →g, `ml`, `l` →ml, `un`) — tabela global, fixa
- **ingredients** — tenant_id, name, category_id, base_unit, min_stock, cost_per_unit, lead_time_days (prazo de reposição), **default_portion** (porção padrão por uso, opcional), active
- **products** — tenant_id, name, category_id, **pricing_mode** (`UNIT` = preço fixo | `WEIGHT` = preço por kg), price, active
- **recipes / recipe_items** — product_id, ingredient_id, quantity (quantidade por unidade do produto). Usado para baixa automática quando o produto tem composição fixa.
- **weight_mix** *(composição média, opcional)* — product_id, ingredient_id, percentage. Ex.: "pote por peso = 70% açaí, 15% creme, 15% complementos". Usado **apenas** para estimativas.

5.3 Estoque (livro-razão)

- **stock_movements** — tenant_id, ingredient_id, type, quantity (+ entrada / − saída), unit_cost, occurred_at, sale_id (opcional), **is_estimated** (bool), note, created_by
  - `type`: `INITIAL` · `PURCHASE` · `SALE_CONSUMPTION` · `MANUAL_OUT` · `LOSS` · `ADJUSTMENT`
- **Saldo** = soma das movimentações por ingrediente (view `stock_levels`). Nunca editamos saldo direto; ajuste é uma movimentação nova → histórico completo e auditável.
- Inventário físico (contagem) gera um `ADJUSTMENT` com a diferença — isso também **corrige o erro das estimativas** periodicamente.

5.4 Vendas

- **sales** — tenant_id, channel (`IN_STORE|ONLINE`), sold_at, total_amount, **total_weight_g** (opcional), notes, created_by
- **sale_items** — sale_id, product_id, quantity, unit_price, weight_g (opcional)
- **sale_item_ingredients** — sale_item_id, ingredient_id, quantity (opcional), **quantity_source**:
  - `MEASURED` — pesado/informado de verdade
  - `RECIPE` — veio da receita fixa
  - `STANDARD_PORTION` — selecionado + porção padrão configurada (estimativa)
  - `UNKNOWN` — sabemos que foi usado, não sabemos quanto

5.5 Suporte

- **audit_log** — tenant_id, user_id, action, entity, entity_id, payload, created_at
- **forecast_snapshots** *(fase 2)* — guarda previsões geradas para medir acurácia depois.

6. Regras de negócio da açaiteria (modeladas de forma genérica)

6.1 Venda presencial (balcão)

- Funcionário registra: **valor**, **peso total do pote** (opcional) e data/hora (padrão: agora).
- Produto do tipo `WEIGHT` (ex.: "Açaí no peso").
- **Não registramos ingredientes** (o caderno/balança não informa).
- Baixa de estoque:
  - Se existir `weight_mix` configurado → gera `SALE_CONSUMPTION` com `is_estimated = true` (peso × percentual).

- Se não existir → nenhuma baixa; a venda conta para demanda de **produto**, e o consumo de ingredientes é reconciliado pelo inventário físico.

6.2 Venda online (site)

- Funcionário registra: **valor**, data/hora e **ingredientes selecionados** pelo cliente.
- Quantidade de cada ingrediente:
  - Com `default_portion` configurada → `STANDARD_PORTION`, baixa estimada.
  - Sem porção → `UNKNOWN`, sem baixa; conta só como "frequência de uso" (útil para ranking e previsão relativa).

6.3 Regra de ouro

> **Ingrediente selecionado ≠ quantidade consumida.** Toda tela que mostra consumo ou saldo derivado de estimativa exibe o selo **"Estimado"**. Saldo real só é confirmado por entrada de compra ou contagem de inventário.

6.4 Venda com receita fixa (outros negócios)

- Produto `UNIT` com receita → baixa automática exata (`RECIPE` , `is_estimated = false`). Ex.: milkshake de 500 ml, sanduíche, kit.

7. Previsão de demanda e reposição (v1)

Objetivo da v1: **simples, explicável e honesta.** Nada de "caixa-preta". Roda dentro da API (TypeScript), sob demanda e com cache diário.

7.1 Requisitos mínimos de dados

Histórico disponível	O que o sistema mostra
< 7 dias	"Histórico insuficiente para previsão." Só mostra totais.
7–27 dias	Previsão por **média móvel** — confiança **baixa**.
≥ 28 dias	Média móvel ponderada **+ fator por dia da semana** — confiança **média**.
≥ 90 dias e baixa variação	Confiança **alta** (avaliar suavização exponencial / Holt-Winters na fase 2).

7.2 Cálculo

1. Série diária de demanda por **produto** (vendas) e por **ingrediente** (consumo registrado/estimado).
2. Média dos últimos N dias, com pesos maiores para dias recentes.
3. Fator de dia da semana = média do dia da semana ÷ média geral (açaí vende mais no fim de semana).
4. Previsão para os próximos 7 dias = média × fator de cada dia.
5. Faixa de incerteza = previsão ± desvio padrão histórico (mostrar como intervalo, nunca número exato).

7.3 Recomendação de reposição (por ingrediente)

```
cobertura          = lead_time_days + dias_de_cobertura_desejados (padrão 7)
demanda_prevista   = soma da previsão diária no período de cobertura
estoque_seguranca  = z × desvio_padrão_diário × √lead_time      (z ≈ 1,65)
sugestao_compra    = max(0, demanda_prevista + estoque_seguranca − saldo_atual)
dias_ate_acabar    = saldo_atual ÷ média_diária
```

Cada sugestão exibe: quantidade, nível de confiança, quantos dias de histórico foram usados e se a base contém consumo estimado.

7.4 Fase 2 (se der tempo)

- Feriados e clima (açaí × temperatura).
- Medir erro das previsões (MAPE) com `forecast_snapshots`.
- Microserviço Python (statsmodels/Prophet) se o volume justificar.

8. Identidade visual

- Fundo **azul-marinho** predominante; textos e elementos em **branco**; um tom de destaque para ações e alertas.
- Paleta inicial (ajustável):
  - `--navy-950: #0A1628` (fundo) · `--navy-900: #0F2040` (cartões) · `--navy-700: #1E3A6B` (bordas/hover)
  - `--white: #FFFFFF` · `--muted: #A9B8D0` (texto secundário)
  - `--accent: #38BDF8` (ações) · `--warning: #F59E0B` (estoque baixo / estimado) · `--danger: #EF4444` · `--success: #22C55E`
- Fonte: **Inter**.
- **Mobile-first**: o registro de venda e de entrada precisa funcionar bem no celular.
- Logo própria "Gestock" (sem marca da açaiteria).

9. Organização Git / GitHub

9.1 Branches

- `main` — sempre estável, protegida (só via PR, CI verde obrigatório).
- `develop` — integração do que está pronto.
- `feat/<escopo>-<descricao>` · `fix/...` · `chore/...` · `docs/...` Ex.: `feat/sales-registro-presencial`

9.2 Commits — Conventional Commits

```
feat(sales): registrar venda presencial com peso
fix(inventory): corrigir arredondamento de saldo
docs: atualizar modelo de dados
```

9.3 Fluxo de trabalho

1. Toda tarefa vira uma **Issue** (com label: `frontend`, `backend`, `banco`, `segurança`, `previsão`).
2. **GitHub Projects** (quadro Kanban): Backlog → A fazer → Em andamento → Revisão → Pronto.
3. Branch a partir de `develop` → PR → revisão de pelo menos 1 colega → merge (squash).
4. Template de PR com checklist (testes, migration, prints de tela).

5. **Milestones** = fases do roadmap abaixo.

10. Roadmap (MVP por fases)

> Ajustar datas conforme o prazo do projeto de extensão.

Fase	Entregas	Itens da especificação
0 — Fundação	Monorepo, lint, CI, banco provisionado, Prisma, design system base, tela de login vazia	—
1 — Contas e isolamento	Cadastro/login, criação de estabelecimento, convite de funcionário, papéis, RLS + teste de isolamento	1, 2
2 — Catálogo	Categorias, ingredientes, produtos (unidade/peso), receitas, mix por peso	3, 4
3 — Estoque	Estoque inicial, compras/entradas, saídas, ajustes, inventário, histórico, saldo, alerta de estoque baixo	5, 6, 10
4 — Vendas	Venda presencial (valor + peso), venda online (ingredientes selecionados), baixa automática/estimada, histórico e filtros	7, 8
5 — Dashboard	Total de vendas, nº de pedidos, mais vendidos, estoque baixo, movimentações recentes	9
6 — Previsão	Previsão de demanda, sugestão de reposição, indicadores de confiança	11
7 — Validação	Seed com dados realistas, uso real na açaiteria, ajustes, deploy, documentação final	—

Fora do MVP: pagamentos, assinaturas, nota fiscal, integração com balança/site, fornecedores (pode entrar como cadastro simples se sobrar tempo).

11. Decisões em aberto (precisamos responder)

1. **Prazo final** do projeto e quantas pessoas no time? (define o tamanho de cada fase)
2. **Banco gerenciado:** Supabase (Postgres) ou Neon? *(sugestão: Supabase)*
3. A açaiteria consegue informar o **peso do pote** em toda venda presencial, ou só o valor?
4. Ela aceitaria definir **porções padrão** (ex.: "1 concha de creme ≈ 40 g") para melhorar as estimativas?
5. Com que frequência dá para fazer **contagem de inventário** (semanal?) — é o que corrige as estimativas.
6. Quantos meses de **histórico do caderno** existem? Vale digitar para alimentar a previsão desde o início.

12. Próximos passos imediatos

1. Revisar este documento juntos e responder a seção 11.
2. Criar o monorepo (fase 0) e o pipeline de CI.
3. Escrever o `schema.prisma` inicial + políticas RLS.
4. Abrir as issues das fases 1 e 2 no GitHub.